

[Home](#)[GitHub](#)

CLASSES

[ArticlesService](#)[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[Articulos](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)

frontend/src/components/atoms/Button.jsx

```
1.  /**
2.   * @file Componente de botón reutilizable.
3.   * @description Un componente de botón versátil con diferentes varia
4.   * @requires react
5.   * @requires prop-types
6.   * @requires ../../styles/atoms/Button.css
7.   */
8.  import React from "react";
9.  import PropTypes from "prop-types";
10. import "../../styles/atoms/Button.css";
11.
12. /**
13.  * @function Button
14.  * @description Renderiza un elemento de botón con estilos y funcion
15.  * @param {object} props - Las propiedades del componente.
16.  * @param {React.ReactNode} props.children - El contenido del botón.
17.  * @param {function} [props.onClick] - Función a ejecutar cuando se
18.  * @param {string} [props.variant='primary'] - La variante de estilo
19.  * @param {string} [props.size='medium'] - El tamaño del botón ('sma
20.  * @param {boolean} [props.fullWidth=false] - Si es `true`, el botón
21.  * @param {boolean} [props.disabled=false] - Si es `true`, el botón
22.  * @param {boolean} [props.loading=false] - Si es `true`, muestra un
23.  * @param {string} [props.type='button'] - El tipo de botón ('button
24.  * @returns {JSX.Element} El componente de botón.
25.  */
26. const Button = ({
27.   children,
28.   onClick,
29.   variant = 'primary',
30.   size = 'medium',
31.   fullWidth = false,
32.   disabled = false,
33.   loading = false,
34.   type = 'button'
35. }) => {
36.   const classNames = [
37.     'btn',
38.     `btn--${variant}`,
39.     `btn--${size}`,
40.     fullWidth ? 'btn--full-width' : '',
41.     loading ? 'btn--loading' : ''
42.   ].filter(Boolean).join(' ');
43.
44.   return (
45.     <button
46.       className={classNames}
47.       onClick={onClick}
48.       disabled={disabled || loading}
49.       type={type}
50.     >
51.       {loading ? (
52.         <span className="btn_loader">
```

```
53.         <span className="btn__spinner"></span>
54.         Cargando...
55.     </span>
56.     ) : children}
57. </button>
58. );
59. };
60.
61. Button.propTypes = {
62.     /** El contenido del botón. */
63.     children: PropTypes.node.isRequired,
64.     /** Función a ejecutar cuando se hace clic en el botón. */
65.     onClick: PropTypes.func,
66.     /** La variante de estilo del botón. */
67.     variant: PropTypes.oneOf(['primary', 'secondary', 'outline']),
68.     /** El tamaño del botón. */
69.     size: PropTypes.oneOf(['small', 'medium', 'large']),
70.     /** Si es `true`, el botón ocupa todo el ancho disponible. */
71.     fullWidth: PropTypes.bool,
72.     /** Si es `true`, el botón está deshabilitado. */
73.     disabled: PropTypes.bool,
74.     /** Si es `true`, muestra un indicador de carga. */
75.     loading: PropTypes.bool,
76.     /** El tipo de botón. */
77.     type: PropTypes.oneOf(['button', 'submit', 'reset'])
78. };
79.
80. export default Button;
```